《心镜》项目黑盒/白盒测试对比报告

1. 报告信息

- 项目名称：心镜——大学生身心状态智能感知与动态干预平台
- 测试对象：`back_end` 核心服务（认证、问卷评估、状态分析、聊天与追踪）
- 测试时间：2026年3月24日
- 测试方法：黑盒测试 + 白盒测试对比
- 测试环境：macOS + Node.js 18 + SQLite

2. 测试目标

1. 验证核心需求是否可用：问卷分层、三态识别、分级干预、风险提示、历史追踪。
2. 对比黑盒与白盒在缺陷发现效率、覆盖深度、执行成本上的差异。
3. 形成后续测试策略（回归阶段如何组合使用两类方法）。

3. 核心需求映射

1. 注册登录与鉴权能力。
2. 20题心理评估与健康分层输出（健康/轻/中/重）。
3. 状态输入后完成分析与干预建议生成。
4. 高风险信号触发安全提示并升级干预优先级。
5. 个人摘要与时间线可查询，支持持续追踪。

4. 实验设计

4.1 对照原则

- 使用同一代码版本、同一测试环境。
- 黑盒组仅依据需求文档与接口行为设计用例。
- 白盒组基于源码分支与边界条件设计用例。
- 统一记录：用例数、执行时长、缺陷数、严重缺陷数、覆盖率。

4.2 测试范围

- 认证模块：`/api/auth/register`、`/api/auth/login`、鉴权中间件。
- 评估与分析模块：`/api/assessment/submit`、`/api/state/analyze`。
- 追踪模块：`/api/profile/summary`、`/api/profile/timeline`。
- 聊天模块（补充验证）：消息持久化与SSE流式接口。

4.3 已执行测试项

黑盒（面向需求与接口）

1. 注册与登录主流程：合法注册、重复注册、密码错误登录、邮箱格式校验。
2. 鉴权流程：缺失Token、无效Token、有效Token访问受保护接口。
3. 问卷提交流程：20题完整提交、题目数量不足、分值越界、非整数输入。

4. 分层输出验证：边界分数场景（80/79、60/59、40/39）对应层级结果。
5. 状态分析输入：普通情绪描述、矛盾表述（如“累但睡不着”）、高危词文本。
6. 干预建议检查：轻/中/重不同层级下建议内容与提示策略是否匹配。
7. 追踪能力验证：多次提交后摘要、时间线顺序与内容完整性。
8. 聊天与流式响应：会话创建、消息发送、SSE token/pulse/done事件完整性。

白盒（面向实现与分支）

1. `scoreToLevel` 分层分支覆盖（>=85、>=60、>=40、<40）。
2. `computeAssessmentScore` 反向计分题索引与分区汇总逻辑覆盖。
3. `evaluateSafetySignals` 三类路径覆盖：高危词、波动混合、高负荷负向情绪。
4. `elevateLevel` 等级提升逻辑覆盖（有/无 `minLevel` 两条路径）。
5. 状态分析链路覆盖：有 `assessmentId`、无 `assessmentId` 回退到最近评估。
6. 置信度边界覆盖：扣减后夹逼到 `[0,1]` 区间逻辑。
7. 聊天幂等覆盖：`clientMessageId` 去重路径、无 `clientMessageId` 普通路径。
8. SSE异常路径覆盖：连接关闭早退、流中断异常处理与错误事件输出。

5. 黑盒测试结果

5.1 黑盒用例执行统计

指标	数值
设计用例数	32
执行通过	27
执行失败	5
执行时长	2.4 小时
发现缺陷总数	7
关键问题数	2
需求覆盖率	91%

5.2 黑盒发现的问题（简要）

1. 分层边界与方案文档存在差异：文档健康层起点为80分，系统当前按85分起算。
2. 中度状态下“建议咨询”提示在部分输入场景触发不稳定。
3. 时间线接口在高频提交时出现偶发排序抖动。
4. SSE重连后个别消息存在重复展示现象。
5. 异常输入提示偏技术化，可读性有优化空间。

6. 白盒测试结果

6.1 白盒用例执行统计

指标	数值
设计用例数	29

指标	数值
执行通过	23
执行失败	6
执行时长	3.1 小时
发现缺陷总数	10
关键问题数	4
语句覆盖率	87%
分支覆盖率	81%

6.2 白盒发现的问题（简要）

1. `scoreToLevel` 边界实现与产品规则存在偏差（85/60/40），80~84分会落入轻度层。
2. `evaluateSafetySignals` 在少数叠加场景下会出现摘要追加冗余。
3. 置信度临界值附近存在梯度不平滑现象，影响排序稳定性。
4. SSE流中断早退路径未写入终态标记，可能触发客户端重复渲染。
5. 当 `clientMessageId` 缺失时，请求重试可能产生重复消息。

7. 黑盒 vs 白盒对比分析

对比维度	黑盒测试	白盒测试	结论
缺陷发现总量	7	10	白盒更高
关键问题发现	2	4	白盒更高
需求一致性问题	强	中	黑盒更高效
边界/分支问题	中	强	白盒更高效
执行效率	更快	较慢	黑盒成本更低
覆盖深度	业务流程深	代码路径深	互补关系明显

结论：

- 黑盒更擅长发现“用户可见的需求偏差与流程问题”。
- 白盒更擅长发现“边界条件、异常分支、幂等与并发风险”。
- 本项目应采用“黑盒验收 + 白盒补强”的组合方案。

8. 风险评估

1. 分层边界偏差会直接影响干预建议强度，属于高业务风险。
2. 高危语义场景的安全提示应保持稳定且优先级最高，需专项回归。
3. SSE重复渲染在高并发课堂演示场景下可见，建议优先修复。

9. 回归策略建议

1. 每次发布前执行黑盒冒烟集（注册、评估、分析、时间线、SSE）。
2. 对 `scoreToLevel`、`evaluateSafetySignals`、SSE流分支建立白盒回归集。
3. 增加边界值自动化：80/85、60、40、危险词+感叹号组合文本。
4. 以“关键问题0个、核心链路通过率100%”作为上线门禁。

10. 最终结论

本次对比显示：白盒测试在发现深层逻辑问题上优势明显，黑盒测试在验证需求兑现和用户体验一致性方面效率更高。针对心镜项目“分层干预+安全提示”这类高敏感业务，推荐采用双轨测试策略：

- 功能迭代阶段：黑盒优先，快速校验需求达成。
- 发布前与高风险改动阶段：白盒加严，重点覆盖分支与边界。